employ a few possible strategies to distribute the computational load:

- Randomly shard the dataset and then independently draw centroids at random from each shard (although it can be tricky to obtain an overall uniform distribution of samples);
- Run canopy clustering first and feed those canopy centroids to k-means as the initial choice of centroids. Then the question becomes: can we also distribute canopy clustering? Although a little trickier, it turns out that we can do that.

It follows that parallelizing canopy clustering is the key step here, and also the trickiest part; we'll worry about it later in this chapter, before delving into this step, let's give a high-level description of the full algorithm for distributed k-means:

· Initialize the centroids using canopy clustering;
· Iterate (at most `m` times)

- **Points classification**:
  - Shard the dataset and send the shards, together with the list of centroids, to mappers;
  - Each mapper assigns points to one of the centroids. Send the data to the reducers, aggregated by the centroid chosen (ideally there will be one reducer per centroid);
- **Centroids update**:
  - Each reducer will compute the center of mass of its cluster and return the new centroid.